LAPORAN PRAKTIKUM
MONGODB PIPLINE AGREGASI

Dosen Pengampu	: M. Harry Khomas Saputra S.T, M.TI.
Nama	: Muhammad Izzudin Irsyad
NIM	: 225443018
Kelas	: 1AEC1

TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

POLITEKNIK MANUFAKTUR BANDUNG

TAHUN AJARAN 2025/2026

DAFTAR ISI

DAFTAR ISI **2**
BAB I PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Tujuan Praktikum 4
BAB II TUTORIAL TERBIMBING **5**
2.1 Setup Lingkungan 5
2.2 Operasi Dasar CRUD 6
2.3 Agregasi via Python 8
2.4 Analisis dengan pandas 9
2.5 Data Generator Langsung 11
2.6 Simulasi Data Sensor Melalui MQTT 12
2.7 Error Handling dan Logging 14
BAB III LATIHAN MANDIRI **16**
3.1 Latihan 1 16
3.2 Latihan 2 19
BAB V KESIMPULAN **23**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Seiring berkembangnya era industri 4.0, pengelolaan data yang cepat, akurat, dan terstruktur menjadi kebutuhan utama dalam sistem manufaktur modern. Data dari mesin produksi, sensor, hingga jadwal perawatan harus dikelola secara efisien agar dapat memberikan wawasan yang berguna bagi pengambilan keputusan. MongoDB, sebagai database NoSQL yang fleksibel dan scalable, menawarkan solusi yang cocok untuk menyimpan data yang bervariasi seperti data sensor, data produksi, hingga data log perawatan. Selain itu, penggunaan pipeline agregasi di MongoDB memungkinkan kita melakukan analisis data kompleks langsung di dalam database tanpa harus memindahkan data ke sistem lain. Praktikum ini disusun untuk memberikan pengalaman langsung dalam mengintegrasikan MongoDB dengan Python, melakukan operasi CRUD, membuat simulasi data sensor, hingga memanfaatkan protokol MQTT untuk komunikasi data real-time. Dengan latar belakang tersebut, praktikum ini menjadi penting untuk membekali saya sebagai mahasiswa Teknologi Rekayasa Informatika Industri dalam menghadapi tantangan pengolahan data di lingkungan industri.

1.2 Tujuan Praktikum

Berdasarkan latar belakang yang telah diuraikan, praktikum ini memiliki beberapa tujuan utama, antara lain:

1. Mampu melakukan setup lingkungan pengembangan dengan virtual environment dan menginstal pustaka yang diperlukan seperti PyMongo, Pandas, Matplotlib, dan Paho-MQTT.
2. Memahami dan mengimplementasikan operasi dasar CRUD (Create, Read, Update, Delete) pada MongoDB menggunakan Python.
3. Menerapkan pipeline agregasi untuk menghitung statistik seperti rata-rata, nilai maksimum, dan jumlah data dari koleksi sensor.
4. Melakukan analisis data sederhana menggunakan Pandas dan visualisasi dengan Matplotlib.
5. Membuat simulasi data generator untuk mengirim data sensor secara langsung ke MongoDB maupun melalui protokol MQTT.
6. Mengimplementasikan error handling dan logging untuk mencatat jalannya proses aplikasi.
7. Menyelesaikan studi kasus mandiri berupa import data dari CSV, query data maintenance, dan simulasi data produksi dengan MQTT.

BAB II
TUTORIAL TERBIMBING

2.1 Setup Lingkungan

Membuat virtual environment, tujuan melakukan ini untuk membuat lingkungan kerja agar terisolasi dari project lain dan mengelola library.

```
python -m venv venv
```

Menginstall pymongo, pandas, python-dotenv, matplotlib, dan paho-mqtt, tujuannya untuk mengakses pustaka yang dibutuhkan sepanjang praktikum ini.

```
PS C:\Users\user\Documents\Nitip (Irsyad)\Polman\Praktik\Sistem Basis Data NoSQL (SBD NoSQL)\Minggu 2\Bahan Praktikum\Hari 2> pip install pymongo panda
s python-dotenv matplotlib paho-mqtt
```

Membuat folder .env, lalu menambahkan program untuk mengakses mongodb local.

```
.env
MONGO_URI=mongodb://localhost:27017
DB_NAME=praktikum6
```

Membuat file koneksi.py, untuk mengkoneksi ke mongodb

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"),
serverSelectionTimeoutMS=5000)
```

```
try:
    client.admin.command('ping')
    print("Koneksi MongoDB berhasil!")
except Exception as e:
    print("Gagal terhubung:", e)
finally:
    client.close()
```

Output:

```
Koneksi MongoDB berhasil!
```

2.2 Operasi Dasar CRUD

Membuat file crud_sensor.py dan mengisinya dengan operasi CRUD

a. Membuat header/setup untuk melakukan operasi CRUD

```
from dotenv import load_dotenv
from pymongo import MongoClient
from datetime import datetime, timedelta
import random, os

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]
```

b. Create (mengisi 10 data sensor)

```
data = []
for i in range(10):
    doc = {
        "mesin": f"CNC-{random.randint(1,3):02d}",
        "suhu": round(random.uniform(60, 100), 2),
```

```
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow() - timedelta(minutes=i*5),
        "status": "normal"
    }
    data.append(doc)
result = collection.insert_many(data)
print(f"Tersimpan {len(result.inserted_ids)} dokumen")
```

Output:

```
ecationWarning: datetime.datetime.utcnow() is deprecated and sc
atetimes in UTC: datetime.datetime.now(datetime.UTC).
  "timestamp": datetime.utcnow() - timedelta(minutes=i*5),
2026-05-05 15:23:12,255 - INFO - Insert berhasil, 10 dokumen
```

c. Read (Data CNC-01)

```
cursor = collection.find(
    {"mesin": "CNC-01"},
    {"_id": 0, "suhu": 1, "timestamp": 1}
).sort("timestamp", -1).limit(5)
print("Data CNC-01 terbaru:")
for doc in cursor:
    print(doc)
```

Output:

```
Data CNC-01 terbaru:
{'suhu': 92.63, 'timestamp': datetime.datetime(2026, 5, 5, 8, 23, 12, 158000)}
{'suhu': 63.72, 'timestamp': datetime.datetime(2026, 5, 5, 8, 13, 12, 158000)}
```

d. Update

```
update_result = collection.update_many(
    {"suhu": {"$gt": 90}},
    {"$set": {"status": "maintenance"}}
)
print(f"Update: {update_result.modified_count} dokumen diubah")
```

Output:

e. Delete

```
satu_jam_lalu = datetime.utcnow() - timedelta(hours=1)
delete_result = collection.delete_many({"timestamp": {"$lt": satu_jam_lalu}})
print(f"Delete: {delete_result.deleted_count} dokumen dihapus")
```

Output:

2.3 Agregasi via Python

Membuat file dan perintah agregasi untuk menghitung rata-rata suhu per mesin, getaran, jumlah data, dan mengurutkannya.

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

pipeline = [
    {"$group": {
        "_id": "$mesin",
        "rata_suhu": {"$avg": "$suhu"},
        "max_getaran": {"$max": "$getaran"},
        "count": {"$sum": 1}
    }},
```

```
    {"$sort": {"rata_suhu": -1}}
]

results = collection.aggregate(pipeline)
print("Rata-rata suhu per mesin:")
for doc in results:
    print(f"{doc['_id']} -> rata:{doc['rata_suhu']:.2f}°C, max getaran:{doc['max_getaran']}, jumlah data:{doc['count']}")

client.close()
```

Output:

```
Rata-rata suhu per mesin:
CNC-03 -> rata:83.12°C, max getaran:0.47, jumlah data:5
CNC-01 -> rata:78.17°C, max getaran:0.4, jumlah data:2
CNC-02 -> rata:72.60°C, max getaran:0.45, jumlah data:3
```

2.4 Analisis dengan pandas

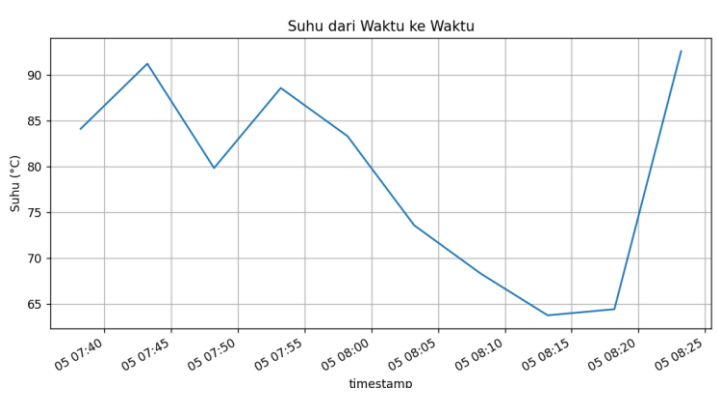
Membuat file dan perintah analisis. Dari perintah berikut, terdapat perintah list(cursor) untuk mengonversi query menjadi list of dicts dan diterima oleh DataFrame. pd.to_datetime() untuk mengubah string atau objek datetime menjadi tipe pandas yang kaya akan fitur. print(df.describe()) untuk menampilkan statistik. Plot grafik untuk memberi visual temporal.

```
import os
import pandas as pd
import matplotlib.pyplot as plt
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
```

```
collection = db["sensor"]

cursor = collection.find({}, {"_id": 0})
df = pd.DataFrame(list(cursor))

df['timestamp'] = pd.to_datetime(df['timestamp'])
df.set_index('timestamp', inplace=True)

resampled = df.resample('10min').mean(numeric_only=True)
print(resampled.head())

plt.figure(figsize=(10,5))
df['suhu'].plot(title='Suhu dari Waktu ke Waktu')
plt.ylabel('Suhu (°C)')
plt.grid(True)
plt.savefig('suhu_plot.png')
plt.show()

client.close()
```

Output:

```
                     suhu  getaran
timestamp
2026-05-05 07:30:00  84.130    0.150
2026-05-05 07:40:00  85.550    0.430
2026-05-05 07:50:00  85.965    0.285
2026-05-05 08:00:00  70.920    0.375
2026-05-05 08:10:00  64.055    0.310
```



2.5 Data Generator Langsung

Membuat data generator yang berjalan dalam infinite loop dan menyisipkan data sensor acak setiap dua detik

```
import time, random
from datetime import datetime
from pymongo import MongoClient
import os
from dotenv import load_dotenv

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

while True:
    doc = {
        "mesin": f"CNC-{random.randint(1,5):02d}",
        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow()
    }
    collection.insert_one(doc)
    print(f"[{datetime.utcnow()}] Inserted: {doc['mesin']} - {doc['suhu']}°C")
    time.sleep(2)
```

Output:

```
t datetimes in UTC: datetime.datetime.now(datetime.UTC).
  print(f"[{datetime.utcnow()}] Inserted: {doc['mesin']} - {doc['suhu']}°C")
[2026-05-05 08:46:15.240002] Inserted: CNC-02 - 66.59°C
[2026-05-05 08:46:17.241695] Inserted: CNC-03 - 66.53°C
[2026-05-05 08:46:19.243696] Inserted: CNC-03 - 60.48°C
[2026-05-05 08:46:21.245409] Inserted: CNC-04 - 66.98°C
[2026-05-05 08:46:23.247136] Inserted: CNC-02 - 94.95°C
```

2.6 Simulasi Data Sensor Melalui MQTT

Menjalankan dua terminal terpisah, satu publiser sensor dan yang satu sebaga subscriber penyimpanan

a. Terminal 1

```
import json, time, random
import paho.mqtt.client as mqtt
from datetime import datetime

BROKER = "broker.hivemq.com"
PORT = 1883
TOPIC = "pabrik/sensor/suhu"

client = mqtt.Client()
client.connect(BROKER, PORT, 60)

while True:
    data = {
        "mesin": f"CNC-{random.randint(1,5):02d}",
        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow().isoformat()
    }
    payload = json.dumps(data)
    client.publish(TOPIC, payload)
    print(f"[PUB] {payload}")
    time.sleep(2)
```

Output:

```
[PUB] {"mesin": "CNC-03", "suhu": 95.78, "getaran": 0.45, "timestamp": "2026-05-05T09:13:21.300561"}
[PUB] {"mesin": "CNC-02", "suhu": 75.18, "getaran": 0.2, "timestamp": "2026-05-05T09:13:23.301998"}
[PUB] {"mesin": "CNC-04", "suhu": 71.2, "getaran": 0.42, "timestamp": "2026-05-05T09:13:25.302795"}
[PUB] {"mesin": "CNC-04", "suhu": 90.78, "getaran": 0.33, "timestamp": "2026-05-05T09:13:27.303615"}
[PUB] {"mesin": "CNC-01", "suhu": 77.05, "getaran": 0.37, "timestamp": "2026-05-05T09:13:29.305049"}
[PUB] {"mesin": "CNC-05", "suhu": 81.81, "getaran": 0.48, "timestamp": "2026-05-05T09:13:31.305922"}
[PUB] {"mesin": "CNC-05", "suhu": 85.97, "getaran": 0.23, "timestamp": "2026-05-05T09:13:33.310599"}
```

b. Terminal 2 (subscriber)

```
import json, os
import paho.mqtt.client as mqtt
from pymongo import MongoClient
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("DB_NAME")

mongo_client = MongoClient(MONGO_URI)
db = mongo_client[DB_NAME]
collection = db["sensor"]

def on_connect(client, userdata, flags, rc):
    print("Terhubung ke MQTT broker")
    client.subscribe("pabrik/sensor/suhu")

def on_message(client, userdata, msg):
    try:
        payload = json.loads(msg.payload.decode())
        payload['timestamp'] = datetime.fromisoformat(payload['timestamp'])
        result = collection.insert_one(payload)
        print(f"[MONGO] Tersimpan {result.inserted_id} -
{payload['mesin']} {payload['suhu']}°C")
            except Exception as e:
                print("Error:", e)
```

```
mqtt_client = mqtt.Client()
mqtt_client.on_connect = on_connect
mqtt_client.on_message = on_message
mqtt_client.connect("broker.hivemq.com", 1883, 60)
mqtt_client.loop_forever()
```

Output:

```
DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  mqtt_client = mqtt.Client()
Terhubung ke MQTT broker
[MONGO] Tersimpan 69f9b6c54e29e484cf4b02f4 - CNC-05 63.23°C
[MONGO] Tersimpan 69f9b6c64e29e484cf4b02f5 - CNC-04 63.65°C
[MONGO] Tersimpan 69f9b6c84e29e484cf4b02f6 - CNC-03 73.72°C
[MONGO] Tersimpan 69f9b6ca4e29e484cf4b02f7 - CNC-05 77.05°C
[MONGO] Tersimpan 69f9b6cc4e29e484cf4b02f8 - CNC-02 81.85°C
[MONGO] Tersimpan 69f9b6ce4e29e484cf4b02f9 - CNC-01 86.84°C
```

2.7 Error Handling dan Logging

Menambahkan logging pada file crud_sensor untuk mencatat proses crud.

```
import logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("crud_sensor.log"),
        logging.StreamHandler()
    ]
)
```

Membungkus proses insert dengan try-except untuk memiliki jejak kronologis ketika aplikasi berjalan di belakang lalu dapat memeriksa di log untuk mendiagnosis masalah

```
try:
    result = collection.insert_many(data)
```

```
    logging.info(f"Insert berhasil, {len(result.inserted_ids)} dokumen")
except Exception as e:
    logging.error(f"Insert gagal: {e}")
```

Output:

```
crud_sensor.log
1   2026-05-05 15:23:12,255 - INFO - Insert berhasil, 10 dokumen
2
```

BAB III

LATIHAN MANDIRI

3.1 Latihan 1

Membuat perintah untuk menghubungkan ke database local latihan6

```
MONGO_URI=mongodb://localhost:27017
DB_NAME=latihan6
```

Membuat data fiktif pada file maintenance.csv untuk dimasukkan ke dalam database

```
mesin,tanggal,biaya,teknisi
CNC-01,2026-05-01,1500000,Zaidan
CNC-02,2026-05-03,750000,Habibie
CNC-01,2026-06-10,1200000,Naufal
CNC-03,2026-06-15,525000,Ica
CNC-02,2026-07-01,250000,Sipa
```

Membuat file untuk mengimport data dari file csv menuju database yang telah dibuat

```
import pandas as pd
from pymongo import MongoClient

# koneksi
client = MongoClient("mongodb://localhost:27017/")
db = client["latihan6"]
collection = db["maintenance"]

# baca CSV
df = pd.read_csv("maintenance.csv")
```

```
# convert tanggal
df["tanggal"] = pd.to_datetime(df["tanggal"])

# ubah ke list of dict
data = df.to_dict(orient="records")

# insert ke MongoDB
collection.insert_many(data)

print("Data berhasil diimport!")
```

Output:

```
Data berhasil diimport!
```

Membuat file query untuk membuat perintah read data biaya lebih dari satu juta, update data nama teknisi dengan biaya 1.2 juta menjadi nama dewi, menghitung total biaya per bulan menggunakan agregasi.

```
import pandas as pd
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["latihan6"]
collection = db["maintenance"]

# # query mencari biaya perawatan lebih dari 1 juta
cursor = collection.find({"biaya": {"$gt": 1000000}}, {"_id": 0})
df = pd.DataFrame(list(cursor))

print(df)
```

```
# # update data teknisi untuk mesin CNC-01 dengan biaya 1.2 juta menjadi Dewi
collection.update_one(
    {"mesin": "CNC-01", "biaya": 1200000},
    {"$set": {"teknisi": "Dewi"}}
)

# # query mencari total biaya perawatan per bulan
pipeline = [
    {
        "$group": {
            "_id": {
                "bulan": {"$month": "$tanggal"},
                "tahun": {"$year": "$tanggal"}
            },
            "total_biaya": {"$sum": "$biaya"}
        }
    },
    {
        "$project": {
            "_id": 0,
            "bulan": "$_id.bulan",
            "tahun": "$_id.tahun",
            "total_biaya": 1
        }
    },
    {
        "$sort": {"tahun": 1, "bulan": 1}
    }
]
```

```
result = list(collection.aggregate(pipeline))
df = pd.DataFrame(result)

print(df)
```

Output:

1. Query Read

```
    mesin    tanggal    biaya teknisi
0  CNC-01 2026-05-01  1500000  Zaidan
1  CNC-01 2026-06-10  1200000  Naufal
```

2. Query Update

```
_id: ObjectId('69fa9dd1fabc90e6e0544359')
mesin : "CNC-01"
tanggal : 2026-06-10T00:00:00.000+00:00
biaya : 1200000
teknisi : "Dewi"
```

3. Query Aggregation

```
   total_biaya  bulan  tahun
0      2250000      5   2026
1      1725000      6   2026
2       250000      7   2026
```

3.2 Latihan 2

Membuat file skrip publisher untuk mengirim data produksi secara acak setiap tiga detik

```
import json, time, random
import paho.mqtt.client as mqtt

BROKER = "broker.hivemq.com"
TOPIC = "pabrik/produksi"

client = mqtt.Client()
client.connect(BROKER, 1883, 60)
client.loop_start()
```

```
while True:
    data = {
        "batch": f"B{random.randint(1,100)}",
        "mesin": f"CNC-{random.randint(1,5)}",
        "jumlah": random.randint(100, 500),
        "reject": random.randint(0, 50)
    }

    client.publish(TOPIC, json.dumps(data))
    print("[PUB]", data)
    time.sleep(3)
```

Output:

```
[PUB] {'batch': 'B31', 'mesin': 'CNC-5', 'jumlah': 209, 'reject': 31}
[PUB] {'batch': 'B31', 'mesin': 'CNC-5', 'jumlah': 278, 'reject': 4}
[PUB] {'batch': 'B44', 'mesin': 'CNC-4', 'jumlah': 176, 'reject': 50}
[PUB] {'batch': 'B14', 'mesin': 'CNC-5', 'jumlah': 377, 'reject': 11}
[PUB] {'batch': 'B21', 'mesin': 'CNC-4', 'jumlah': 261, 'reject': 45}
[PUB] {'batch': 'B89', 'mesin': 'CNC-5', 'jumlah': 219, 'reject': 31}
[PUB] {'batch': 'B16', 'mesin': 'CNC-2', 'jumlah': 235, 'reject': 9}
[PUB] {'batch': 'B7', 'mesin': 'CNC-4', 'jumlah': 170, 'reject': 14}
[PUB] {'batch': 'B89', 'mesin': 'CNC-3', 'jumlah': 123, 'reject': 40}
[PUB] {'batch': 'B48', 'mesin': 'CNC-5', 'jumlah': 202, 'reject': 16}
[PUB] {'batch': 'B47', 'mesin': 'CNC-2', 'jumlah': 352, 'reject': 7}
```

Membuat file dan perintah subscriber yang menerima dari publish lalu menyimpan dalam mongodb di koleksi produksi_mqtt. Dalam subscriber, menghitung reject rate.

```
import json
import paho.mqtt.client as mqtt
from pymongo import MongoClient
from datetime import datetime

client_mongo = MongoClient("mongodb://localhost:27017/")
db = client_mongo["latihan6"]
collection = db["produksi_mqtt"]
```

```
def on_connect(client, userdata, flags, rc):
    print("Connected:", rc)
    client.subscribe("pabrik/produksi")

def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())

    # tambah timestamp
    data["timestamp"] = datetime.now()

    # hitung reject rate
    reject_rate = (data["reject"] / data["jumlah"]) * 100

    if reject_rate > 5:
        print("⚠ WARNING: Reject tinggi!")
        data["peringatan"] = True
    else:
        data["peringatan"] = False

    collection.insert_one(data)
    print("[SUB]", data)

mqtt_client = mqtt.Client()
mqtt_client.on_connect = on_connect
mqtt_client.on_message = on_message

mqtt_client.connect("broker.hivemq.com", 1883, 60)
mqtt_client.loop_forever()
```

Output:

```
[SUB] {'batch': 'B31', 'mesin': 'CNC-5', 'jumlah': 278, 'reject': 4, 'timestamp': datetime.datetime(2026, 5, 6, 9, 50, 59, 799575), 'peringatan': False,
id': ObjectId('69faac9390 6f005950fd1c7c')}
⚠WARNING: Reject tinggi!
[SUB] {'batch': 'B44', 'mesin': 'CNC-4', 'jumlah': 176, 'reject': 50, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 2, 838932), 'peringatan': True, '
d': ObjectId('69faac9690 6f005950fd1c7d')}
[SUB] {'batch': 'B14', 'mesin': 'CNC-5', 'jumlah': 377, 'reject': 11, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 5, 799230), 'peringatan': False,
id': ObjectId('69faac9990 6f005950fd1c7e')}
⚠WARNING: Reject tinggi!
[SUB] {'batch': 'B21', 'mesin': 'CNC-4', 'jumlah': 261, 'reject': 45, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 8, 828702), 'peringatan': True, '
d': ObjectId('69faac9c 906f005950fd1c7f')}
⚠WARNING: Reject tinggi!
[SUB] {'batch': 'B89', 'mesin': 'CNC-5', 'jumlah': 219, 'reject': 31, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 11, 946773), 'peringatan': True,
id': ObjectId('69faac9f 906f005950fd1c80')}
[SUB] {'batch': 'B16', 'mesin': 'CNC-2', 'jumlah': 235, 'reject': 9, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 14, 869809), 'peringatan': False,
id': ObjectId('69faaca2 906f005950fd1c81')}
⚠WARNING: Reject tinggi!
[SUB] {'batch': 'B7', 'mesin': 'CNC-4', 'jumlah': 170, 'reject': 14, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 17, 838711), 'peringatan': True, '
d': ObjectId('69faaca5 906f005950fd1c82')}
⚠WARNING: Reject tinggi!
[SUB] {'batch': 'B89', 'mesin': 'CNC-3', 'jumlah': 123, 'reject': 40, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 20, 962110), 'peringatan': True,
id': ObjectId('69faaca8 906f005950fd1c83')}
⚠WARNING: Reject tinggi!
[SUB] {'batch': 'B48', 'mesin': 'CNC-5', 'jumlah': 202, 'reject': 16, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 23, 868712), 'peringatan': True,
id': ObjectId('69faacab 906f005950fd1c84')}
[SUB] {'batch': 'B47', 'mesin': 'CNC-2', 'jumlah': 352, 'reject': 7, 'timestamp': datetime.datetime(2026, 5, 6, 9, 51, 26, 835738), 'peringatan': False,
id': ObjectId('69faacae 906f005950fd1c85')}
```

BAB V
KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, saya dapat menyimpulkan beberapa hal berikut:

1. Penggunaan virtual environment berhasil mengisolasi proyek dan instalasi pustaka seperti pymongo, pandas, matplotlib, serta paho-mqtt berjalan lancar.
2. Operasi CRUD pada MongoDB melalui PyMongo dapat diimplementasikan dengan baik, meliputi pembuatan data acak, pembacaan dengan filter, pembaruan status mesin, serta penghapusan data lama.
3. Pipeline agregasi efektif digunakan untuk menghitung rata-rata suhu, nilai maksimum getaran, dan jumlah data per mesin secara langsung di database.
4. Integrasi dengan Pandas memungkinkan analisis lebih mendalam seperti resampling data time-series, sedangkan Matplotlib membantu visualisasi tren suhu.
5. Simulasi data generator berhasil berjalan secara real-time, baik langsung ke MongoDB maupun melalui protokol MQTT dengan bantuan broker HiveMQ.
6. Penerapan error handling dan logging membuat sistem lebih mudah dilacak dan di-debug.
7. Pada latihan mandiri, saya berhasil mengimpor data CSV, melakukan query dan update data maintenance, menghitung total biaya per bulan, serta membuat sistem publisher-subscriber untuk data produksi lengkap dengan peringatan reject rate.

Secara keseluruhan, praktikum ini memberikan pemahaman yang baik tentang integrasi MongoDB, Python, dan MQTT untuk membangun sistem pengelolaan data industri yang real-time dan skalabel.